

# 整改清单

7.3.

作品报告前两页直接用Word转PDF插入即可

## 参考文献

直接放在后面

正式的论文放在后面

github放在脚注

## 小标题注意一下

除了摘要和参考，都要有第几章

注意页眉不要分开，换成作品的名字，居中

## 摘要

摘要过长，稍微凝练一下，不需要太具体，摘要强调目的，而不是技术手段

把1,2,3,4缩进去

落脚到做到了什么，以及这样的好处，强调已经做出来了什么，好处是什么，具体路径手段一笔带过

摘要和下面的2,3有重复的地方

不要写具体的攻击复杂度，直接写难以找到碰撞

直接说目前作品的创新点（做成了什么），不要罗列具体的技术

不要写奇怪的东西，如“本作品实现了用户端、骑手端和管理员端的完整业务闭环”，不要让别人一眼看出摘要要是AI生成的

订单通信越权改为订单隐私泄露

背景动机做了什么有什么好处

=====

**关键词：**外卖平台；可控匿名；订单认证；PID，HMAC-SM3；Merkle Tree；条件溯源；可信审计

这两个不能作为关键词，建议删掉 ✓

## 作品概述

### 1.1.3 国密算法推广与相关法律法规背景

在外卖订单通信安全问题的技术层面，密码算法的选择至关重要。长期以来，我国信息系统中广泛使用 RSA、AES、SHA-256 等国际密码算法，但这些算法存在被植入后门或存在未公开弱点等潜在风险。为保障国家信息安全，我国大力推进商用密码算法的自主研发和推广应用。

这个地方RSA，SHA-256，往政策合规方面写 ✓

## 1.2 相关工作与现有方案不足

针对外卖订单通信中的安全问题，学术界和产业界已开展但现有方案在系统性、安全性和适用性方面仍存在不足。

现有外卖平台的隐私保护措施。以美团、饿了么为代表的号码、隐私面单等隐私保护功能。例如，美团规则中心规定商家

---

6 《中华人民共和国密码法》 主席令第三十五号 2019年10月26日公布

这里用空格，不要用句号 ✓

平台治理改为“平台管理” ✓

“了”日常匿名、异常可溯源”的

注意引号的方向 ✓

**解耦就是降低系统或模块之间的依赖关系，使各部分能够独立运行和维护。**

### 基本概念

解耦（Decoupling）指的是让系统中的各个模块\*\*“松散地连接”\*\*，尽量减少相互依赖，使修改或扩展某个模块时不会影响其他模块，从而避免“牵一发而动全身”的复杂关联  CSDN+1。通俗理解就是两个模块不直接交互，而是通过中间媒介传递信息，这样一个模块的变动不会直接影响另一个模块  CSDN。

解耦，不要出现 ✓

## 预备知识

设  $H$  表示密码杂凑函数,  $K$  表示共享密钥,  $m$  表示待认证消息, 则 HMAC 的一般形式可以表示为:

$$\text{HMAC}(K, m) = H((K' \oplus \text{opad}) \parallel H((K' \oplus \text{ipad}) \parallel m)) \quad (1)$$

其中,  $K'$  表示经过填充或压缩处理后的密钥 (若密钥长度小于杂凑函数的分组长度, 则在右侧填充零; 若大于分组长度, 则先对密钥计算一次杂凑值),  $\text{opad}$  (outer pad) 和  $\text{ipad}$  (inner pad) 分别表示外部填充值和内部填充值,  $\parallel$  表示字符串连接操作,  $\oplus$  表示按位异或。对于输出 256 bit 摘要的杂凑函数 (如 SM3),  $\text{ipad}$  为字节 0x36 重复 32 次构成的 512 bit 序列,  $\text{opad}$  为字节 0x5C 重复 32 次构成的 512 bit 序列。

HMAC 的安全性取决于底层杂凑函数的性质和密钥的保密性。在公式化安全公式后面的解释不要换行 ✓

第十六面的三个复杂度删掉 ✓

以 CBC 模式为例, 加密时需要引入初始化向量  $IV$ , 使相同明文在不同加密过程中得到不同密文, 从而降低明文模式泄露风险。其抽象形式如下:

$$C = \text{SM4-CBC-Enc}_K(IV, M) \quad (8)$$

具体而言, CBC 模式的加密过程为:  $C_0 = IV$ ,  $C_i = \text{Enc}_K(M_i \oplus C_{i-1})$ , 其中  $M_i$  和  $C_i$  分别表示第  $i$  个明文分组和密文分组。解密过程为:  $M_i = \text{Dec}_K(C_i) \oplus C_{i-1}$ 。由于每个分组的加密都依赖于前一个密文分组, 攻击者无法仅通过替换某个密文分组来篡改对应的明文而不影响后续分组的解密结果。

直接说用的是CBC, 不要“以CBC”为例 ✓

其中,  $K_{\text{master}}$  为平台主密钥,  $userID$  为用户真实身份标识,  $r$  为随机数。引入随机数  $r$  的作用在于实现**跨订单不可链接性** (Cross-Order Unlinkability): 即使同一用户在不同订单中使用 PID, 由于每次生成的随机数不同, 攻击者无法通过比较

19

跨订单不可连接改为避免敌手实现并发攻击 ✓

订单通信过程可以抽象为:

$$Client \xrightarrow{orderID, credential} Server \quad (10)$$

$$Server \xrightarrow{\text{verify}(orderID, credential)} Room(orderID) \quad (11)$$

$$Message \xrightarrow{\text{forward}} User/Rider \quad (12)$$

这个公式改为图三那样的, 画成示意图 ✓

图 4: Merkle Tree 结构示意图 (任意叶子变化均导致 Root 改变)

Merkle Tree 的一个重要优势是支持高效的**包含证明** (Inclusion Proof): 要验证某个叶子节点  $leaf_i$  是否包含在以  $root$  为根的 Merkle Tree 中, 验证者只需知道从  $leaf_i$  到  $root$  路径上的兄弟节点哈希值 (即验证路径), 即可通过  $O(\log n)$  次哈希计算完成验证, 而无需获取全部  $n$  个叶子节点。这一性质使得 Merkle Tree 在大规模数据集的完整性验证中具有显著的效率优势。

在 FoodShield 系统中, Merkle Tree 用于订单通信日志的完整性验证。系统并不直接将明文消息作为叶子节点, 而是将与消息相关的摘要信息作为叶子节点输入。为了绑定订单上下文、发送方角色、消息顺序和密文内容, 本系统中单条消息日志的叶子节点可以抽象定义为:

包含证明改为成员关系证明 (member ship proof) ✓

## 系统设计

P29面表格的分号改为顿号 ✓

3.6 数据库设计

FoodShield 使用 SQLite 作为数据存储后端，数据库共设计四张核心表：users (用户表)、orders (订单表)、messages (消息表) 和 audit\_logs (审计日志表)。各表字段的设计均遵循”最小必要原则”，避免将超出业务所需的敏感信息明文保存在数据库中。

用户表 (users)

用户表存储用户的注册信息与匿名身份标识，字段设计如表 5 所示。

FoodShield

外卖订单隐私认证与可信审计系统

表 5: users 表字段说明

| 字段名           | 类型      | 说明            | 安全设计意义                 |
|---------------|---------|---------------|------------------------|
| id            | INTEGER | 自增主键          | 内部索引，不对外暴露             |
| username      | TEXT    | 用户名           | 用于登录认证                 |
| pid           | UNIQUE  |               |                        |
|               | TEXT    | 匿名身份标识        | 隐藏真实 id，用于通信           |
|               | UNIQUE  | (PID)         | 和溯源                    |
| pid_r         | TEXT    | PID 生成随机数 $r$ | 仅服务端保存，用于条件溯源时重算 PID   |
| password_hash | TEXT    | 口令哈希值         | SM3(salt    password), |

下面表格的字体换成上面的打字机字体 ✓

## 表关系与隐私设计原则

四张表之间通过外键关联，整体以订单号为核心索引构建数据流。`users` 与 `orders` 通过 `user_id` 关联，`orders` 与 `messages` 通过 `order_id` 关联，`audit_logs` 通过 `order_id` 与订单和消息建立审计关系。

在隐私保护方面，数据库设计遵循以下原则：其一，消息明文不落库，`messages.content` 字段存储 SM4-CBC 密文，明文只在服务端内存中短暂存在；其二，`users.pid_r`（随机数  $r$ ）与 `users.pid` 均仅服务端维护，前端不可见，骑手端不可见；其三，`orders.note` 备注字段在 API 响应中对骑手端和管理员端屏蔽，仅在用户本人查询订单时返回；其四，`messages.sender_pid` 字段中存储的是 PID 而非用户名，管理员默认视图也不直接暴露 PID 对应的真实用户名。

40

写具体的是哪几张表 ✓

数据库内部：`users.pid_r`（随机数  $r$ ）和 `users.pid` 均仅存于服务端数据库，仅服务器进程可访问，不通过 API 直接对外暴露。

## 跨订单不可链接性

由于每位用户的 PID 由随机数  $r$  参与生成，不同用户之间的 PID 不存在可预测的关联关系。骑手在多次接单过程中，即使多次与同一用户通信，在缺少  $K_{\text{master}}$  和  $r$  的情况下，也无法通过 PID 判断是否为同一用户。HMAC-SM3 的伪随机性确保：在  $K_{\text{master}}$  保密的前提下，已知 PID 无法还原 `userID`，也无法枚举其他用户的 PID，从而实现跨订单不可链接性。

## 3.9 订单 Token 认证机制

### 机制概述

订单 Token 是用户进入订单通信通道前必须提交的认证凭证。Token 由服务器在订单创建时生成，并与订单绑定保存。用户发起 WebSocket 入房请求时，服务器对 Token 的合法性、归属关系和时效性进行三重验证，缺一不可。

44

改为跨订单攻击 ✓

P45页有超行 ✓

图片九（47页）箭头重叠了，剪头不要超过方框

骑手端入房（`join_order_as_rider` 事件）需提交：

$$\{ order\_id, role \}$$

骑手端不需要提交 Token, 但服务器会检查订单状态, 仅当订单处于 `taken`、`delivering` 或 `completed` 状态时, 才允许骑手加入对应房间。这一状态约束保证只有已接单的骑手才能进入订单通信通道, 避免任意骑手随意侦听订单房间。

### 消息转发与字段最小化

用户端或骑手端通过 `send_message` 事件发送消息时, 消息负载包含：

$$\{ order\_id, sender\_pid, role, content \}$$

P48改为字母改成打印体 ✓

- **实时快照**: 每条消息发送成功并写入数据库后, 服务器立即调用 `create_merkle_snapshot` 函数, 以当前全部消息哈希重新计算 Root, 将  $\langle order\_id, MERKLE\_ROOT\_UPDATED, root, message\_count \rangle$  写入 `audit_logs`;
- **管理员手动快照**: 管理员可通过 `POST /admin/snapshot/{order_id}` 接口手动触发某订单的 Merkle Root 快照;

右边不要换行, 已经超过文本区域了 ✓

## G2：认证性

**目标：**只有持有合法 Token 的订单参与方才能进入对应订单通信通道，攻击者无法伪造或复用 Token 冒充合法用户。

**分析：**Token 公式为  $\text{HMAC-SM3}(K_{\text{order}}, \text{orderID} \parallel \text{PID} \parallel \text{timestamp} \parallel \text{nonce})$ 。在  $K_{\text{order}}$  保密的前提下，HMAC 的不可伪造性保证攻击者无法在不知道密钥的情况下构造任何有效 Token。时间戳有效期（默认 3600 秒）限制了截获 Token 后的可利用时间窗口；nonce 的引入确保即使时间戳相同，重放攻击也无法复现合法 Token。

**复杂度估计：**在随机预言模型下，HMAC-SM3 满足 EUF-CMA（存在不可伪造性），伪造成功概率不超过  $q/2^{256}$ ，其中  $q$  为查询次数。

57

G1之类的，改成目标一，直接使用文本 ✓

作品实现提到git仓库，最好是放个链接 ✓

## 作品实现

73面图片过大，缩小，图片的字体小于这个正文的字体 ✓

## 功能展示与创新性说明

几个创新的小标题太人机了，而且这一张不涉及到功能展示  
“场景创新”改为“贴近生活的高频应用场景” ✓

## 最后一张

加一个现有不足，未来工作 ✓

## 附录

加一个项目结构 ✓

每一张下面的附录不要超过10个